

Playwright Automation Bootcamp

Session 1 – Playwright Automation Bootcamp

NODE.JS

TYPESCRIPT

AUTOMATION STRATEGY



Your Playwright Automation Bootcamp: 3.5 Days at a Glance

Session	Theme	Core Focus
Session 1	Modern Testing & Language Foundations	Node.js, TypeScript, automation strategy, Playwright architecture
Session 2	Playwright Core: Setup, Locators & First Tests	Browser/context/page lifecycle, locators, auto-wait, Lab 1
Session 3	Stability & Scalable Test Design	Assertions, synchronisation, Page Object Model, fixtures
Session 4	Hybrid Testing: Playwright Across Layers	APIRequestContext, API preconditioning, login bypass, hybrid Lab
Session 5	BDD, Full API Automation & Mocking	Cucumber, API CRUD suite, network interception, Mokapi
Session 6	Visual Regression, Tooling & Framework Architecture	Snapshots, Trace Viewer, CI/CD, Allure, framework scaffold
Session 7	Beyond UI: Rest Assured, Performance & Security	Rest Assured, Gatling, OWASP, unified strategy

 Sessions 1–6 each include a hands-on lab. Session 7 is a demo and discussion session.

What You Will Be Able to Do

→ Playwright's Role

Articulate where Playwright fits in a modern automation strategy

→ Test Pyramid

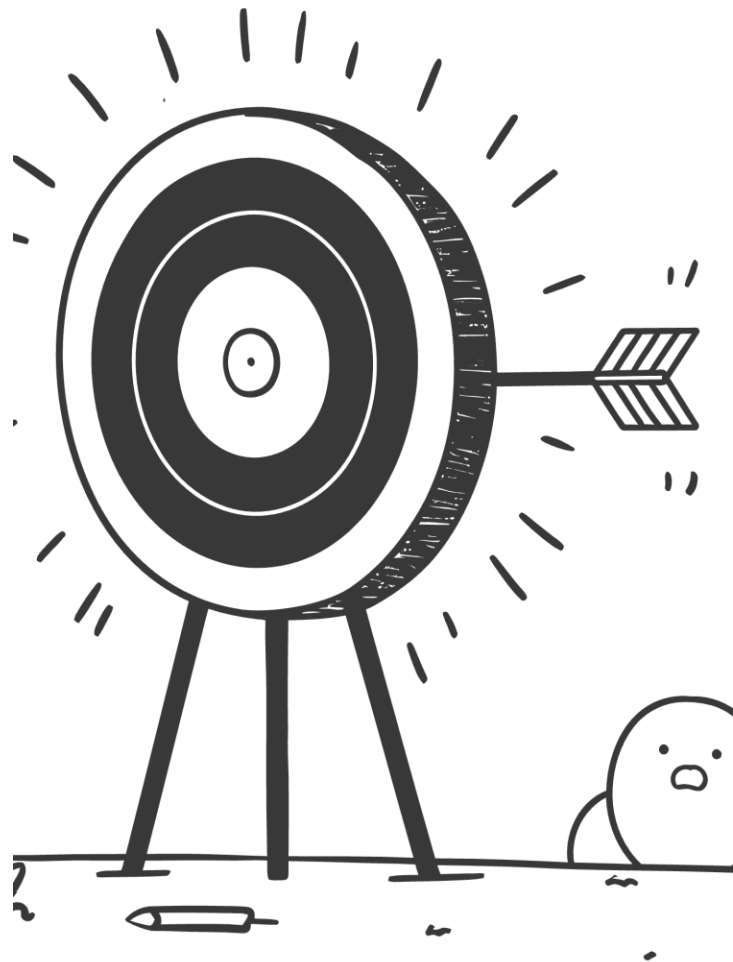
Explain when UI automation is the right choice

→ Node.js / TypeScript Setup

Set up and run a basic project from scratch

→ TypeScript Fluency

Write types, interfaces, async/await, and configure tsconfig.json



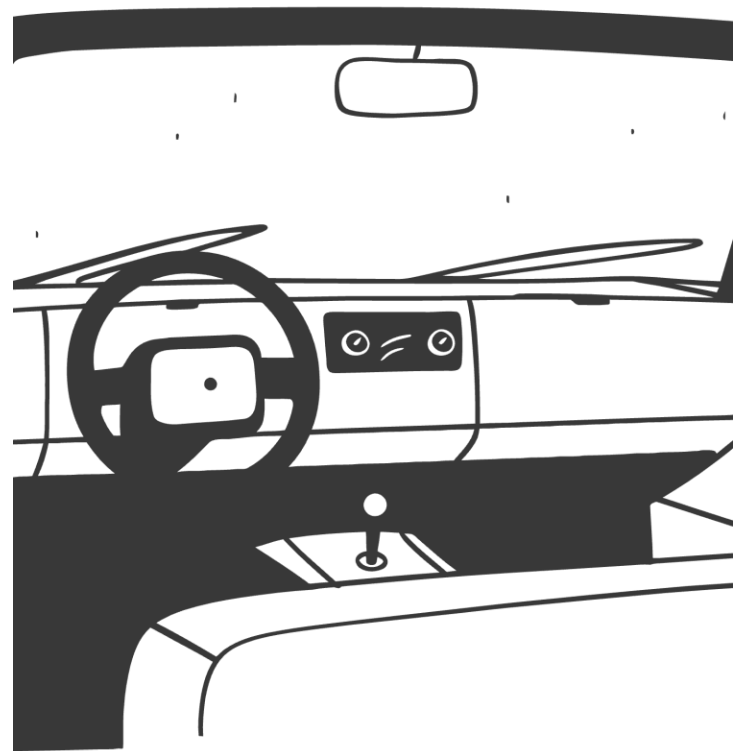
Testing Has Changed — Has Your Approach?

Then

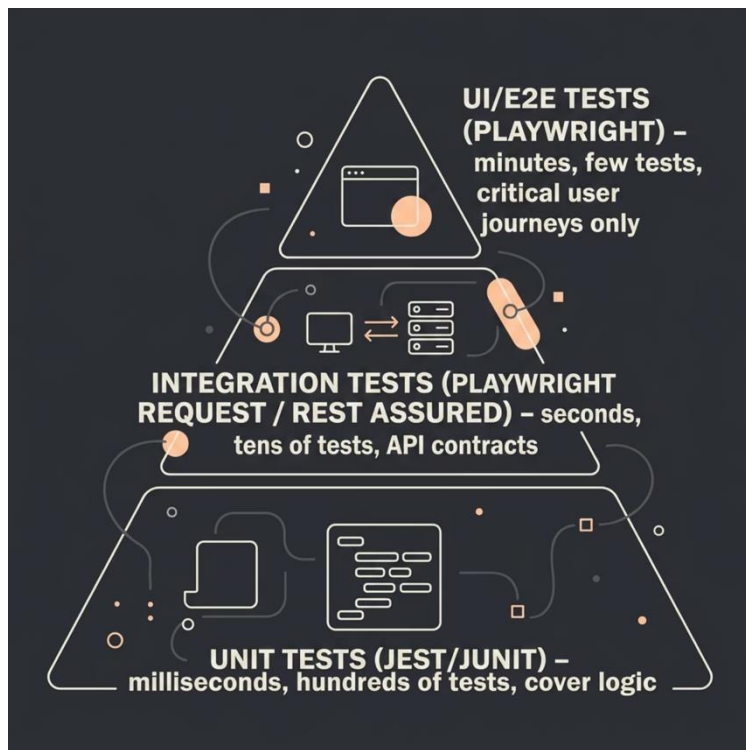
- Manual regression cycles of days or weeks
- QA as a phase at the end
- Automation as an afterthought
- Defects found late = expensive to fix

Now

- Continuous testing in CI/CD
- QA engineers are automation engineers
- Code-first, maintainable frameworks
- Defects caught in minutes, not weeks



Test at the Right Layer



The Golden Rule

UI tests are expensive — automate only journeys that represent genuine user risk and cannot be tested at a lower layer.

Unit

Milliseconds · Hundreds of tests · Cover logic

Integration

Seconds · Tens of tests · API contracts

UI / E2E

Minutes · Few tests · Critical journeys only

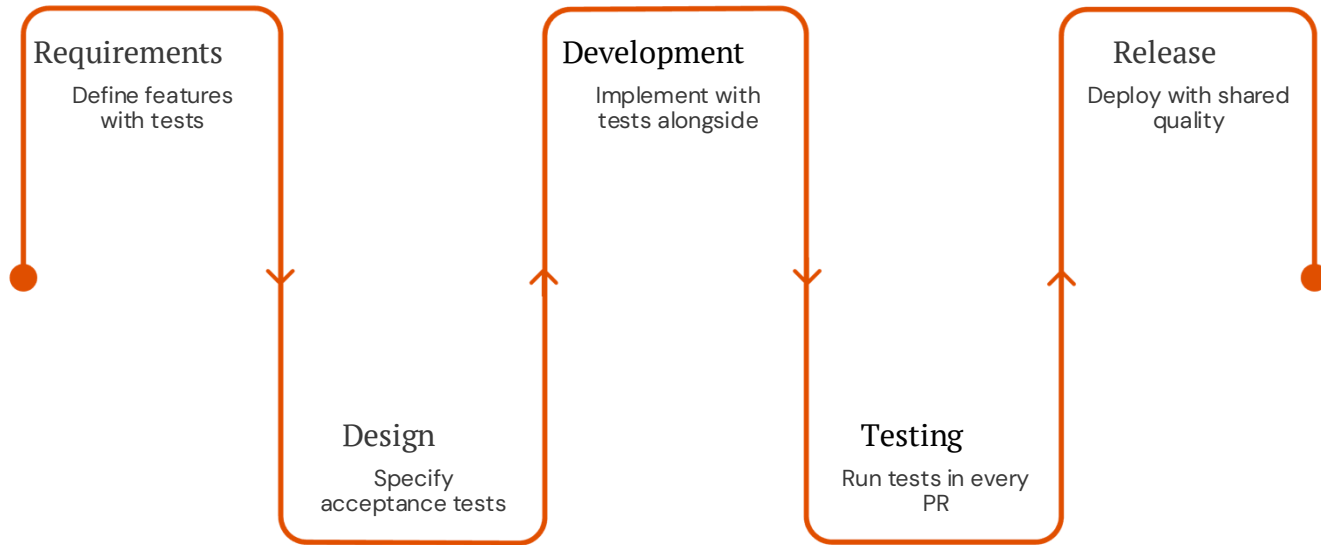
10x

Cheaper to Fix in Development Than in Production

A defect found during development costs **10 times less** to fix than one discovered in production. Shift left isn't just good practice — it's good economics.



Shift Left: Embed Testing Earlier



1

Write tests as acceptance criteria

Before development starts

2

Run tests in every PR

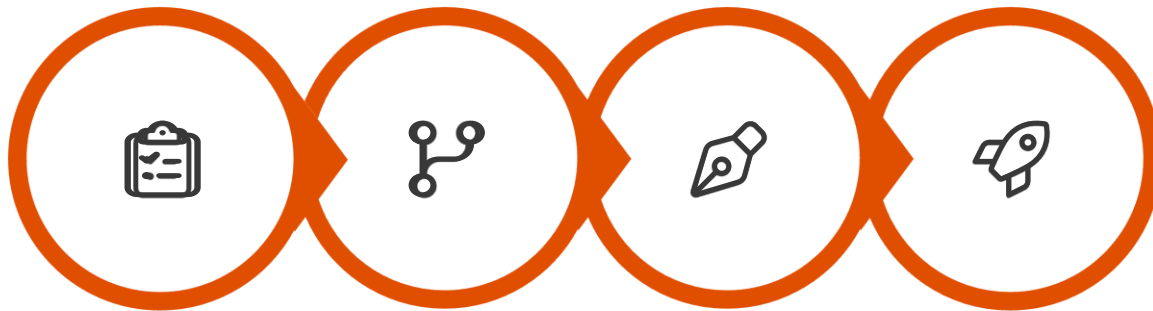
Not just before release

3

Developer + QA collaboration

Shared ownership of quality

Shift Left: Embed Testing Earlier



Requirements

Development

Tests During Dev

Release

Acceptance Criteria First

Write tests as acceptance criteria before development starts

Every Pull Request

Run tests in every PR, not just before release

Dev + QA Collaboration

Testers review code; developers review test scenarios

Playwright Fundamentals

BROWSER

CDP

WEBDRIVER



Playwright Architecture



Playwright uses **CDP (Chrome DevTools Protocol)** directly — not WebDriver — making it faster and more reliable than Selenium.

Browser

One process per browser type

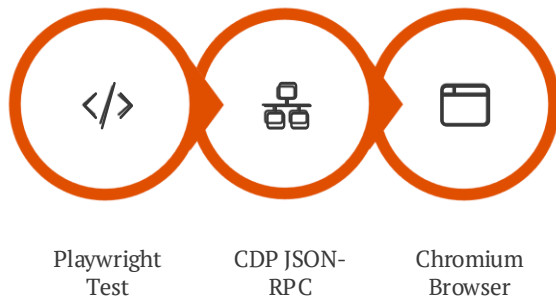
Browser Context

Isolated session: cookies, storage, auth state

Page

One tab per test

Chrome DevTools Protocol: Playwright's Direct Line to the Browser



What CDP Exposes

DOM

Inspect & manipulate elements

Network

Intercept & mock requests

Runtime

Execute JavaScript in-browser

Emulation

Device & geolocation emulation

📌 CDP is the same protocol used by Chrome DevTools (F12). Playwright is essentially scripting against the DevTools API.

Why CDP Outperforms WebDriver for Modern Test Automation

Dimension	WebDriver (Selenium)	CDP (Playwright)
Protocol	W3C WebDriver — HTTP REST	JSON-RPC over WebSocket
Connection	New HTTP request per command	Single persistent WebSocket
Intermediary	ChromeDriver / GeckoDriver binary	None — direct to browser
Speed	Slower — TCP + HTTP overhead	Faster — low-latency messages
Browser Events	Poll-based (retry until condition)	Event-driven (subscribe to events)
Network Control	Limited — no native interception	Full — intercept, modify, mock
Auto-wait	Manual sleeps or FluentWait	Built-in actionability events

 WebDriver's W3C standardisation ensures broad compatibility. CDP's directness gives Playwright its speed, reliability, and advanced features like network mocking and device emulation.

Why Playwright Over Selenium or Cypress?

Feature	Selenium	Cypress	Playwright
Cross-browser	Via WebDriver	Chromium only	Chromium + Firefox + WebKit
API + UI in one test	No	Limited	Yes
Auto-wait	Manual waits	Yes	Yes
Parallel execution	External setup	Limited	Built-in
Network interception	No	Yes	Yes
Speed	Slow	Fast	Fast
Language support	Many	JS/TS only	JS/TS + Python + Java + C#

Part A — Key Takeaways

01

Playwright wins

Cross-browser, built-in parallelism, API + UI in a single framework

02

Test Pyramid guides decisions

Reserve UI tests for genuine user-risk journeys only

03

Shift left saves money

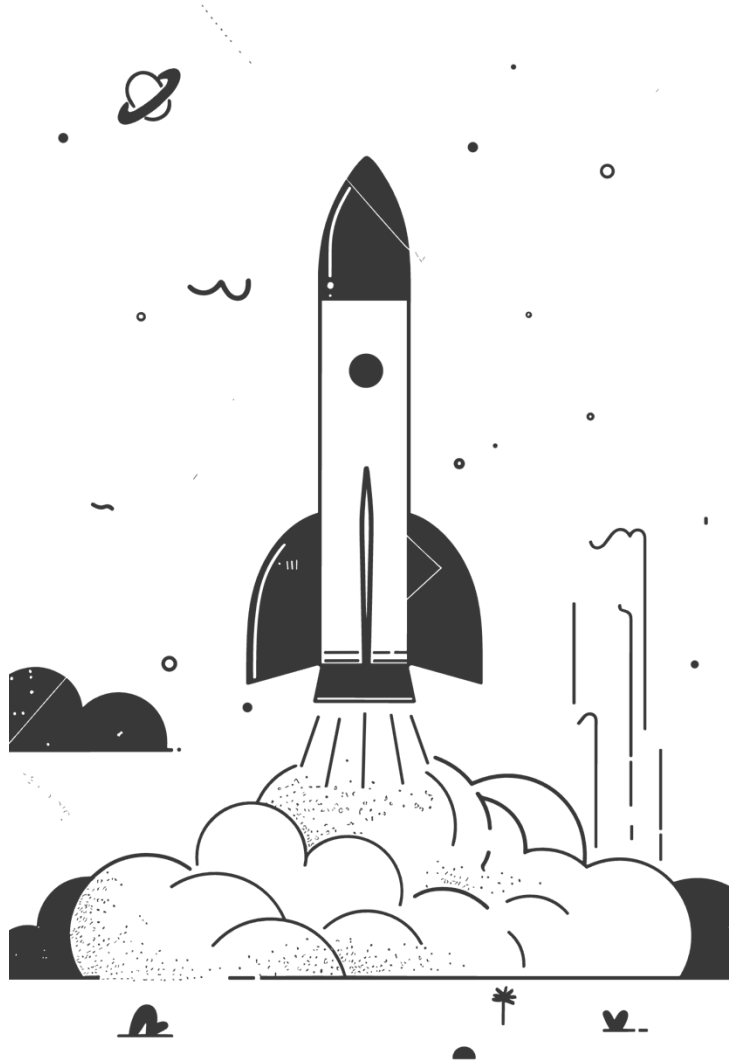
Defects fixed in development cost 10x less than in production

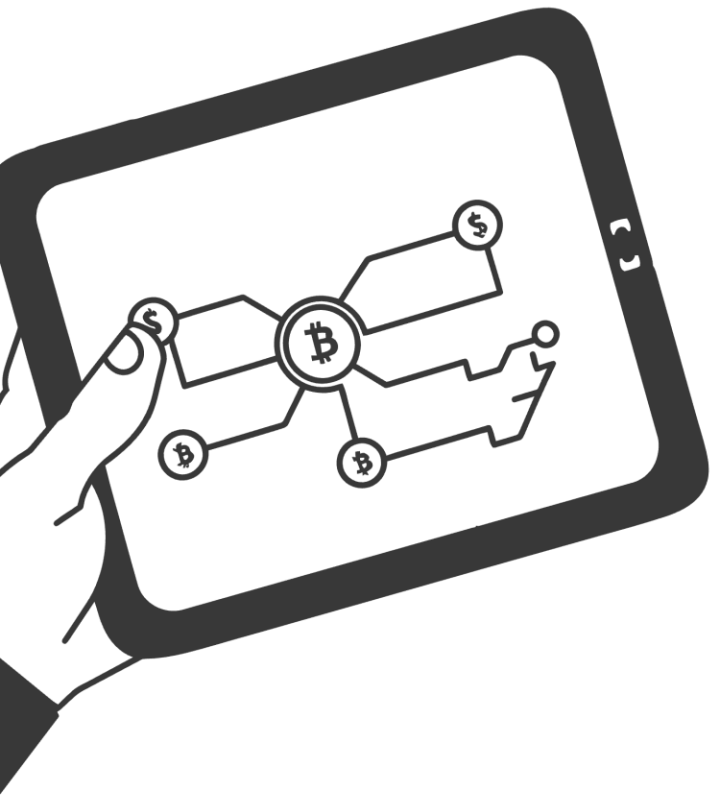
04

TypeScript is foundational

Types, interfaces, and strict mode make automation maintainable at scale

📌 Up next: Part B — Playwright Core Concepts & Writing Your First Tests





PLAYWRIGHT AUTOMATION BOOTCAMP

Session 1 Part B — Modern Testing & Language Foundations

What Is Node.js?

The Runtime Behind Playwright

Node.js runs JavaScript **outside the browser**, built on Chrome's V8 engine. It uses an event-driven, non-blocking I/O model.

Playwright is a Node.js library — test files are Node.js processes that control browsers.



Node uses a **single-threaded event loop** vs Java's multi-threaded blocking model.

Verify Your Setup

```
node -v  
# e.g. v20.11.0
```

```
npm -v  
# e.g. 10.2.4
```

```
node -e "console.log('Hello from Node')"
```


NPM and package.json

NPM

Node Package Manager — installs and manages all libraries

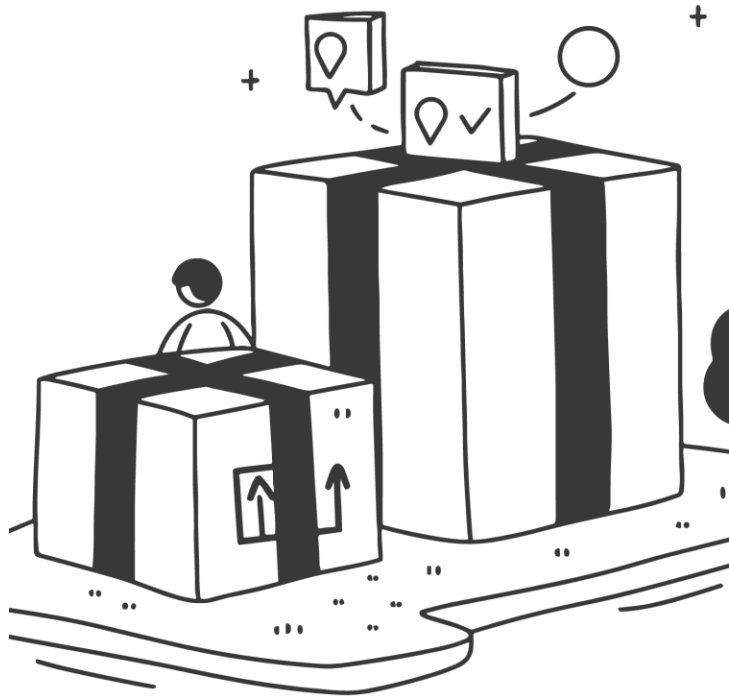
package.json

Project manifest: dependencies, devDependencies, scripts, metadata

node_modules/

Where installed packages live — **never commit to git**

```
"test": "playwright test"  
"test:headed": "playwright test --headed"  
"report": "playwright show-report"
```



Node.js & TypeScript Project Setup

Project Scaffold

```
npm init -y
npm install typescript ts-node --save-dev
npx tsc --init
```

Key tsconfig.json Settings

```
{
  "target": "ES2020",
  "module": "commonjs",
  "strict": true,
  "outDir": "./dist"
}
```

Why TypeScript?

- Catch type errors at compile time, not runtime
- Interfaces enforce consistent test data shapes
- Async/await makes test flow readable
- IDE autocomplete accelerates authoring

 Strict mode is strongly recommended for test automation projects.

TypeScript Essentials for Test Automation



Types & Interfaces

Define shapes for test data, page objects, and API responses to prevent runtime surprises



Async / Await

Write asynchronous browser interactions in a clean, readable, sequential style



Modules & Imports

Organize helpers, fixtures, and page objects into reusable, maintainable modules



Strict Mode

Enforce null checks and explicit types across the entire test codebase

TypeScript Fundamentals

Why TypeScript?

- Type errors caught at **compile time**, not runtime
- VS Code autocomplete and IntelliSense
- Self-documenting: types explain what a function expects

Before vs. After

```
// JavaScript – no safety  
function login(username, password) { }
```

```
// TypeScript – compile-time safety  
interface UserCredentials {  
  username: string;  
  password: string;  
}  
function login(creds: UserCredentials) { }
```

async/await for Testers

Why async?

Browser interactions — clicking, navigating, waiting — are asynchronous. Test code must **await** each step before asserting.

✅ **Golden rule:** Every Playwright API call that touches the browser must be awaited.

Clean, readable test code

```
// With async/await – reads top-to-bottom  
await page.goto('/login');  
await page.fill('#username', 'user');  
await page.click('button');
```

```
// Without – nested .then() chains  
// hard to read, hard to debug
```

tsconfig.json

Configuring TypeScript for a Playwright project — generated by `npm init playwright@latest`. When in doubt, use the generated defaults.



`"target": "ES2020"`

Output JS version — supports async/await natively



`"module": "commonjs"`

Required for Node.js



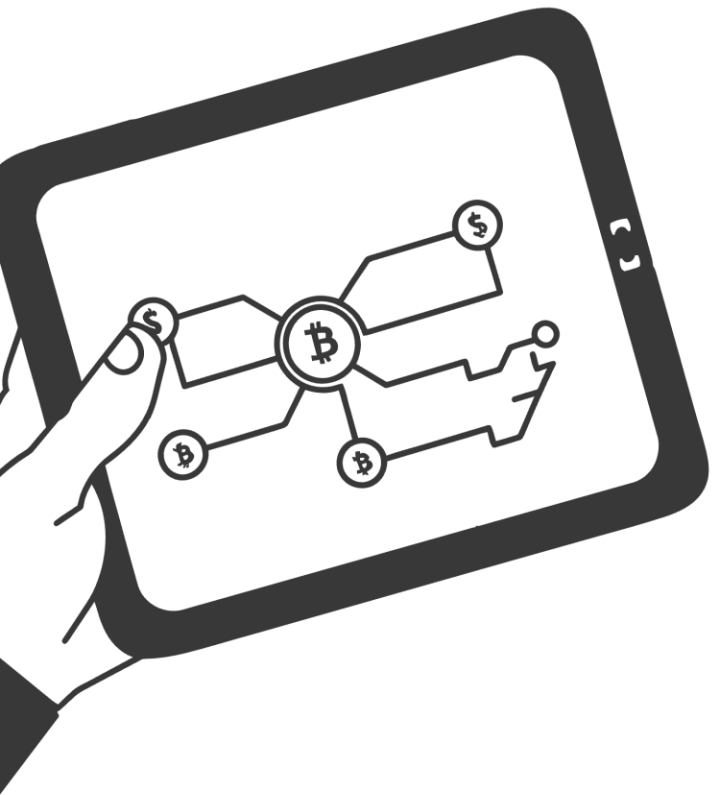
`"strict": true`

Enables all type checks, catches more bugs



`"include": ["tests/**/*.ts"]`

Only compile test files



PLAYWRIGHT AUTOMATION BOOTCAMP

Labs

<https://github.com/ashishjuyal/db-pw-may-26-batch-2>

Hands-On Exercise

Quick Exercise

Build a typed TypeScript helper file from scratch, practising every language feature you will use when writing Playwright tests from Session-2 onwards:

1. Scaffold a TypeScript project (NPM + tsconfig)
2. Write basic types: string, number, boolean, array
3. Define interfaces for test data shapes
4. Write typed functions with parameters and return types
5. Write and call an async/await function
6. Convert three JavaScript snippets to TypeScript
7. Run a full type check and fix all errors



Follow GitHub instructions for completing the lab.

Meet TestMart: Your Practice Application

A realistic Node.js e-commerce app. Not a toy — with async interactions, REST APIs, authentication, and edge cases built to teach real automation challenges. *Setup in Session 2. Watch the demo now.*

Key Pages to Automate

Home / Products	Navigation, async search, filters
Login	Forms, errors, locked accounts
Cart / Checkout	Updates, validation, order placement
Orders	History and confirmation

Three User Accounts

Standard User	Default UI tests
Admin	Create & manage products via API
Locked User	Tests locked-account error state

What Makes It Realistic

400ms Search Debounce

Visible spinner — tests that ignore it are flaky

Async Cart Updates

Button changes to "Adding..." before reverting

Toast Notifications

Auto-dismiss after 3 seconds

Out-of-Stock Product

Product #12 has zero stock; Add to Cart absent

JWT Auth + REST API

Bypass login UI; hybrid UI + API testing from Session 4

 12 seed products across Electronics, Accessories & Software. **Session 1 lab:** Build a TypeScript data factory for the product catalogue — before touching the browser.



Session 1 Complete: What You've Built

01

Automation Strategy

Test Pyramid and Shift Left principles

02

Playwright Architecture

Browser, context, and page isolation

03

Node.js & NPM

Project setup and dependency management

04

TypeScript

Types, interfaces, async/await, tsconfig.json

What's Next

1

Install & Scaffold

Set up Playwright and your first project

2

First UI Test

Write a test from scratch; master locator strategies

3

Interactions

Inputs, dropdowns, and dialogs

4

Lab 1

Automate Login → Search → Logout flow

 **Action before Session 2:** Run `node -v` (must return 20+) and install the **VS Code Playwright extension**.